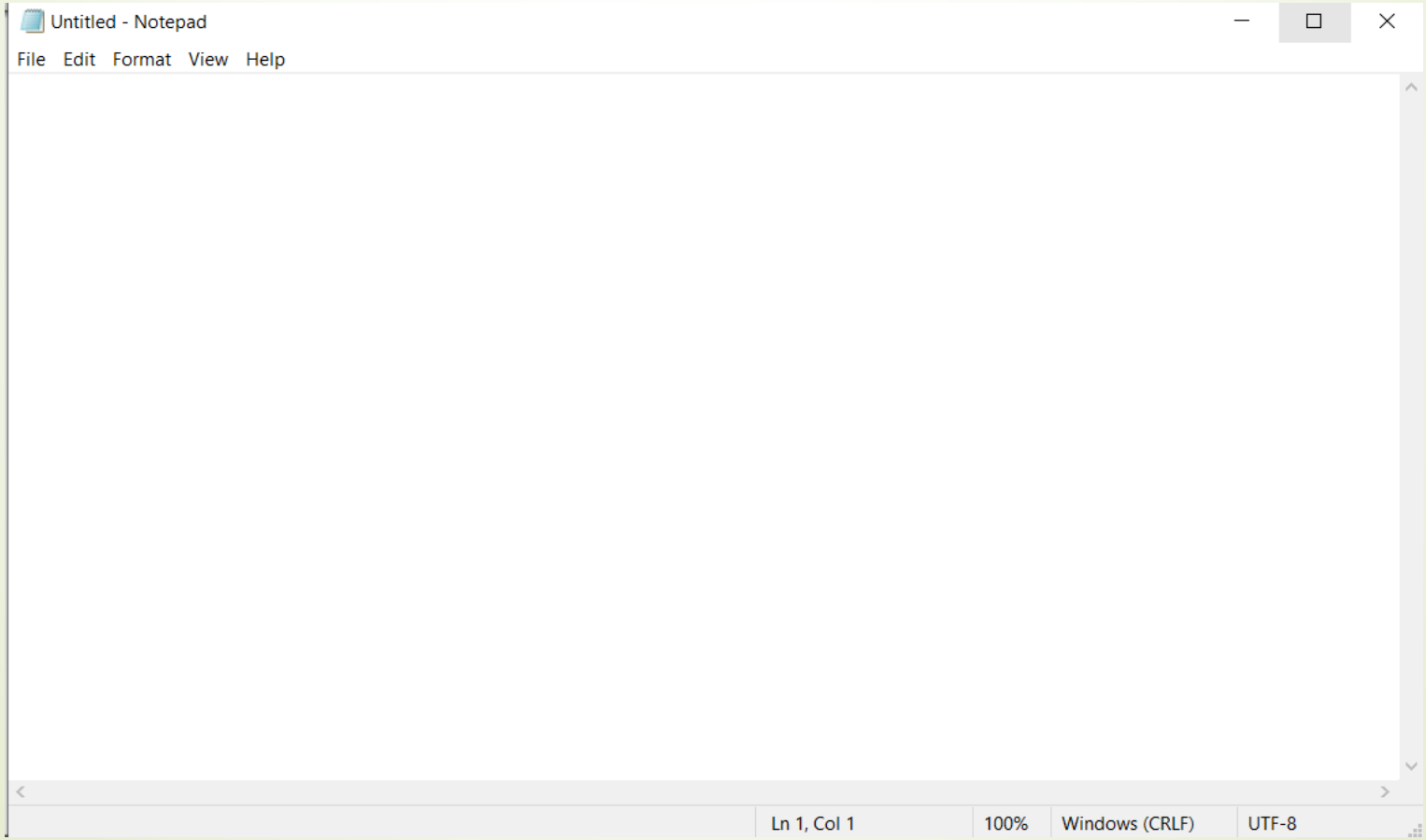
Lecture_2

Windows Programming_II

Dr. Sara Mohamed

Single-document interface (SDI) applications.

- These applications allow a user to work with only one window at a time.



Multiple-document interface (MDI) applications

- Several large Windows applications, such as Microsoft Excel and Visual Studio .NET, allow users to work with several open windows at the same time for example Photoshop application.
- The main application window of an MDI application acts as the **parent window**, which can open several child window.
- **Multiple-document interface (MDI)** applications enable you to display multiple documents at the same time, with each document displayed in its own window. MDI applications often have a Window menu item with submenus for switching between windows or documents.

- The main application window of an MDI application acts as the parent window, which can open several child windows. You need to know the following main points about an MDI application:
  - Child windows are restricted to their parent window (that is, you cannot move a child window outside its parent window).
  - The parent window can open several types of child windows. As an example of an MDI application, Visual Studio .NET allows you to work with several types of document windows at the same time.
  - The child windows can be opened, closed, maximized, or minimized independently of each other, but when the parent window is closed, the child windows are automatically closed.

- The MDI frame should always have a menu; one of the menus that a user always expects to see in an MDI application is the Window menu, which allows the user to manipulate various windows that are open in an MDI container form.

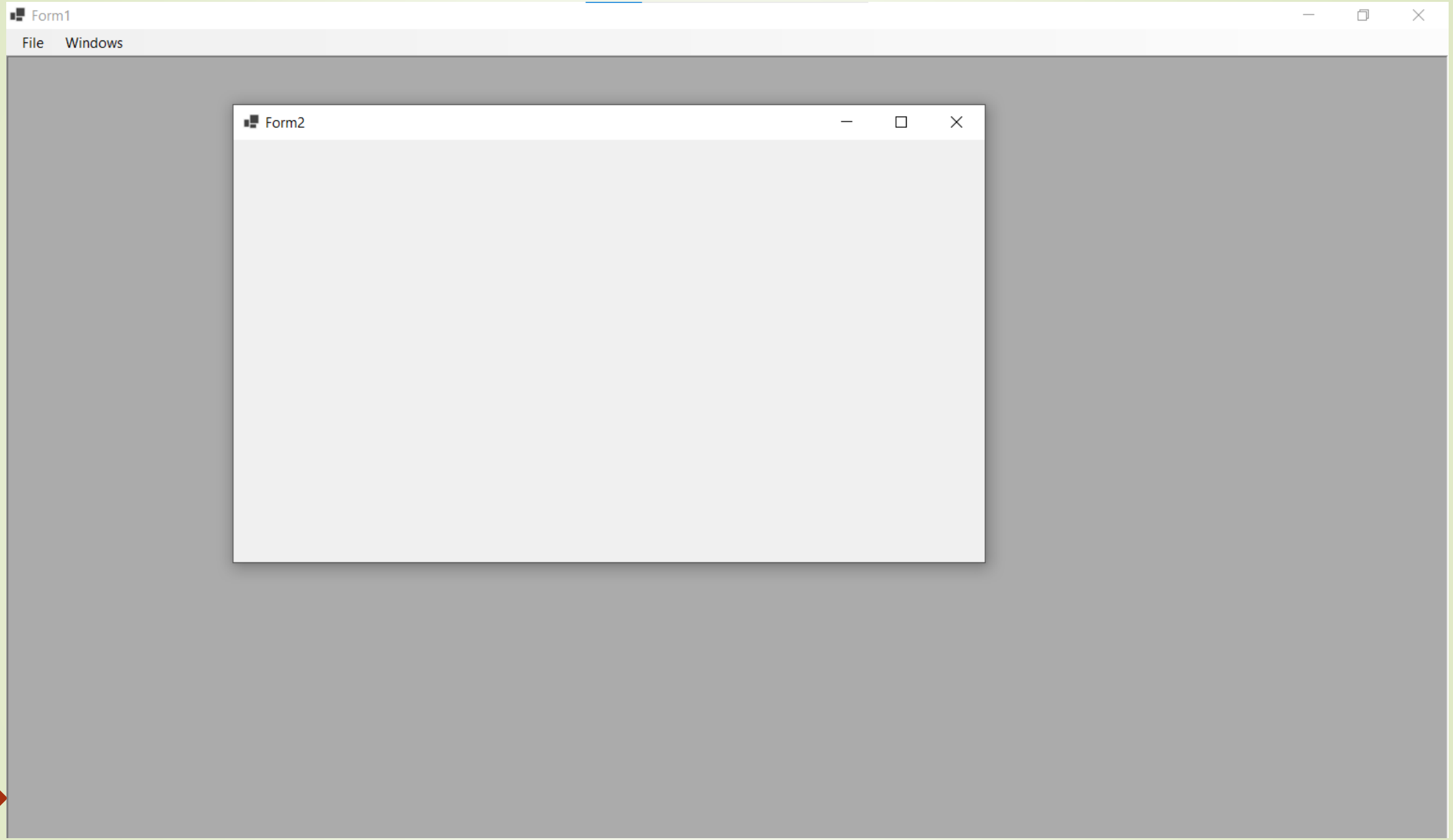
IsMdiContainer property to true

- The Windows Forms in an MDI application are also created by using the standard System.Windows.Forms.Form class. To create an MDI parent form, you create a regular Windows form and change its **IsMdiContainer** property to true.

Important Members of the Form Class That Are Related to MDI Applications

Member	*Type*	*Description*
`ActiveMdiChild`	Property	Identifies the currently active MDI child window
`IsMdiContainer`	Property	Indicates whether the form is a container for MDI child forms
`MdiChildActivate`	Event	Occurs when anMDI child form is activated or closed within an MDI application
`MdiChildren`	Property	Specifies an array of forms that represent the MDI child form of the form
`MdiParent`	Property	Specifies the MDI parent form for the current form
`LayoutMdi()`	Method	Arranges the MDI child forms within an MDI parent form

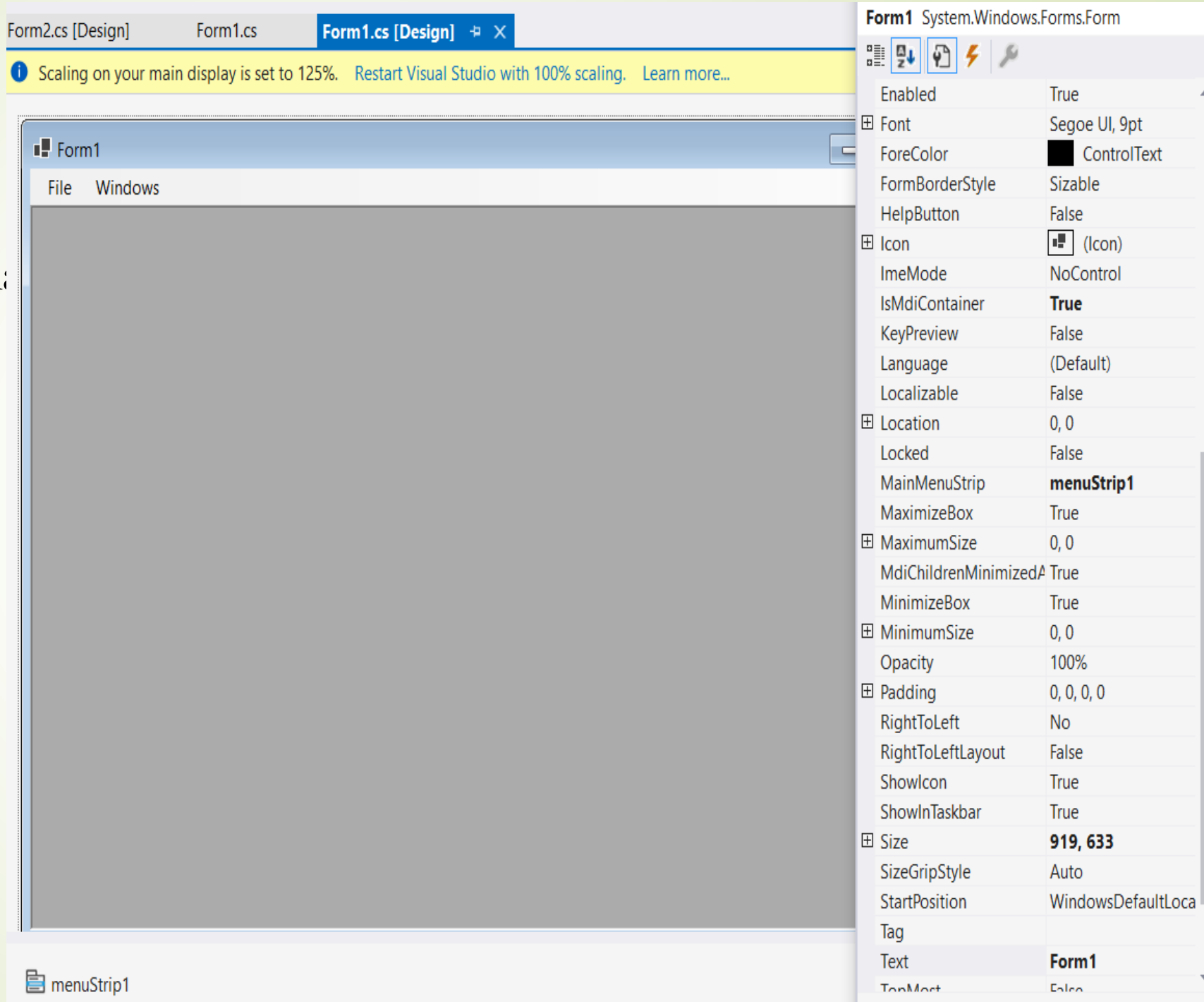
Example_1


Form1
File
Windows
Form2

1. Add Menustrip to Form1 window, then add item and ma the text property it to File .

2. make the isMdiContainer property for the form to True

3. make the WindowState to Maximize



🟩 2️⃣ Set from Properties Window (Visual Studio)

1. Select the Form
2. Go to **Properties**
3. Find **WindowState**
4. Choose **Maximized**

✅ No code needed 👍

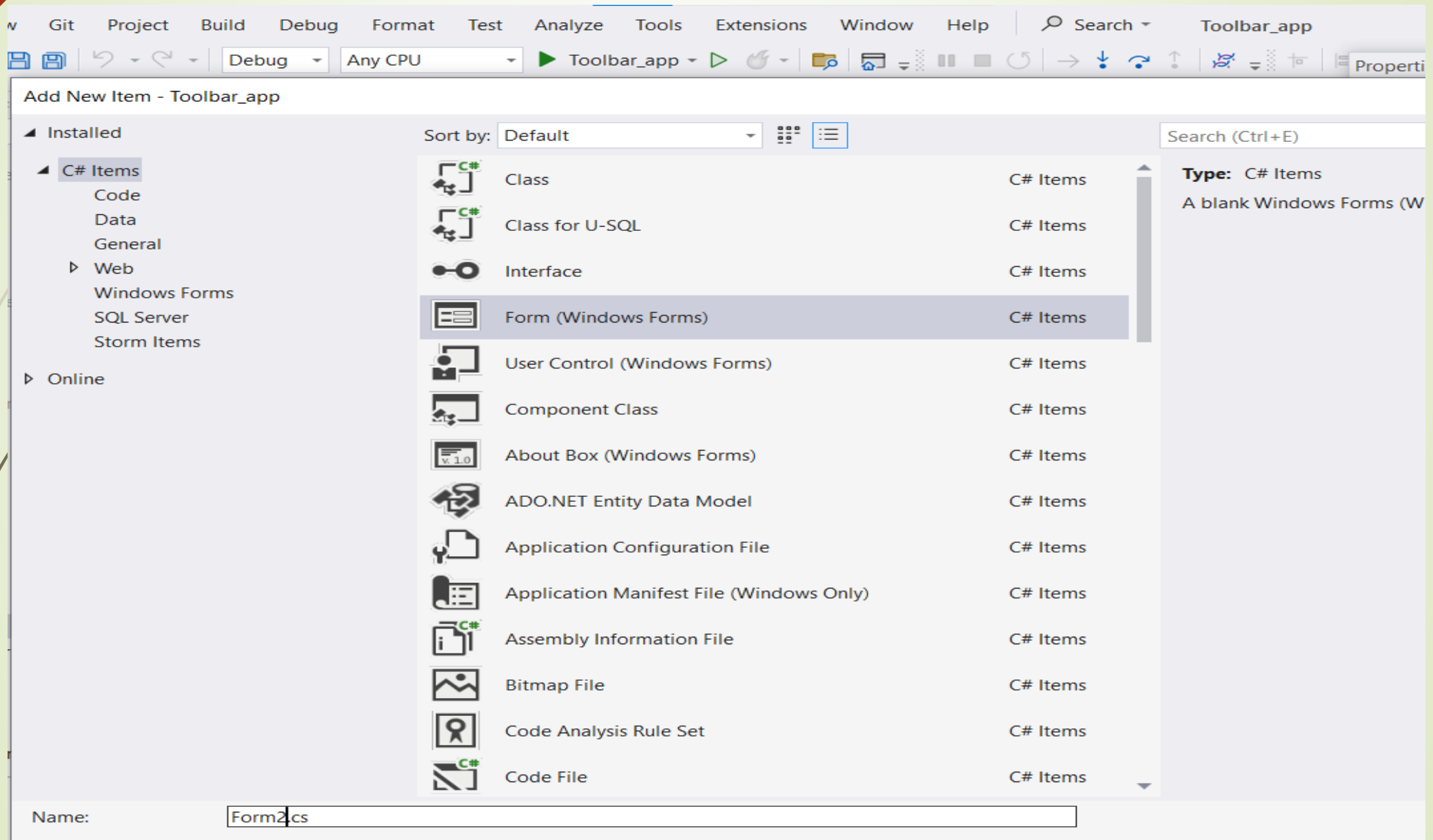
🟨 3️⃣ WPF (C#)

```C#
this.WindowState = WindowState.Maximized;
```

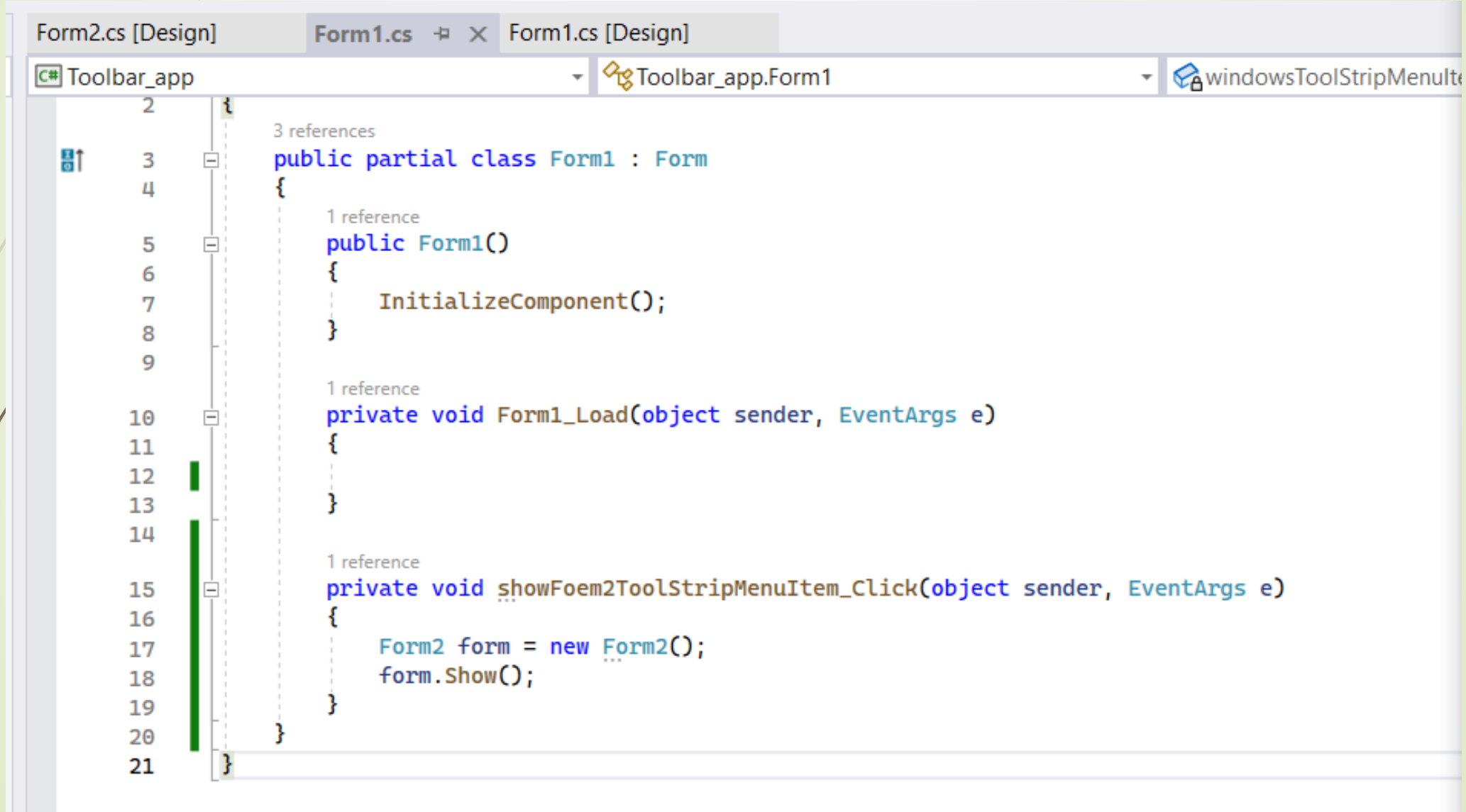
📌 Result

The window will open **maximized automatically** 🔲➡️💻

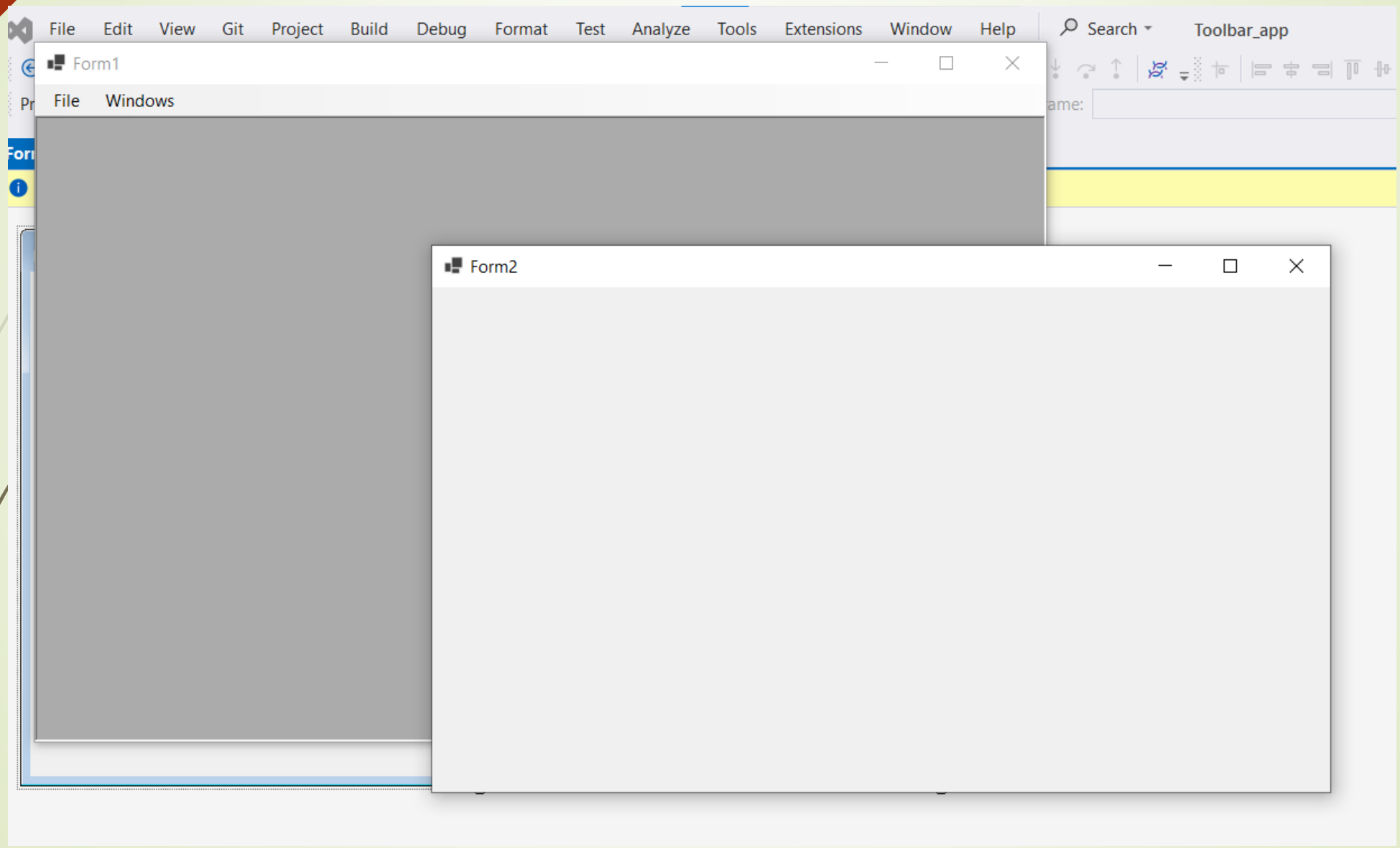
Add new windows form from Project tab



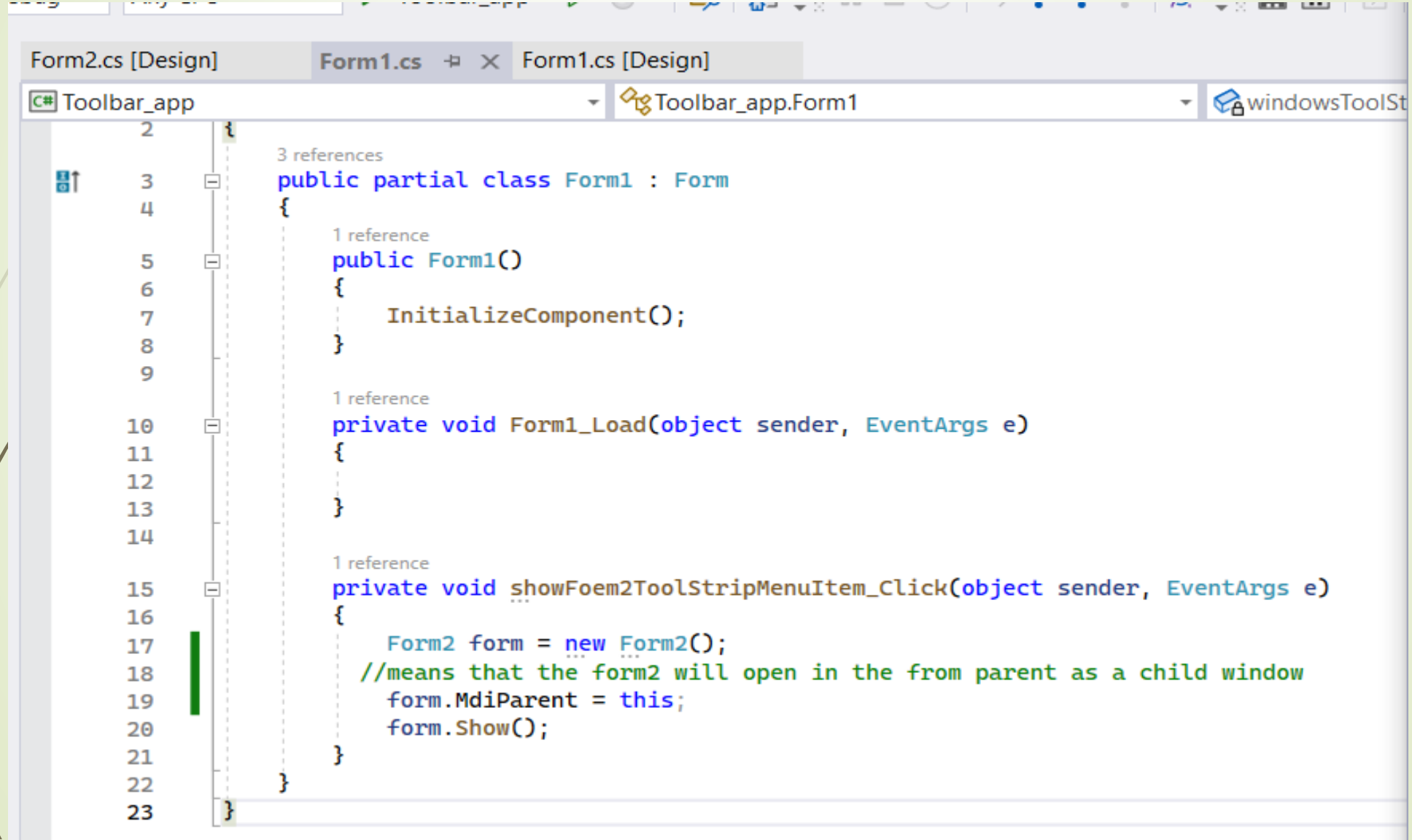
Open the click event for the ToolStripMenuItem then type the following code:

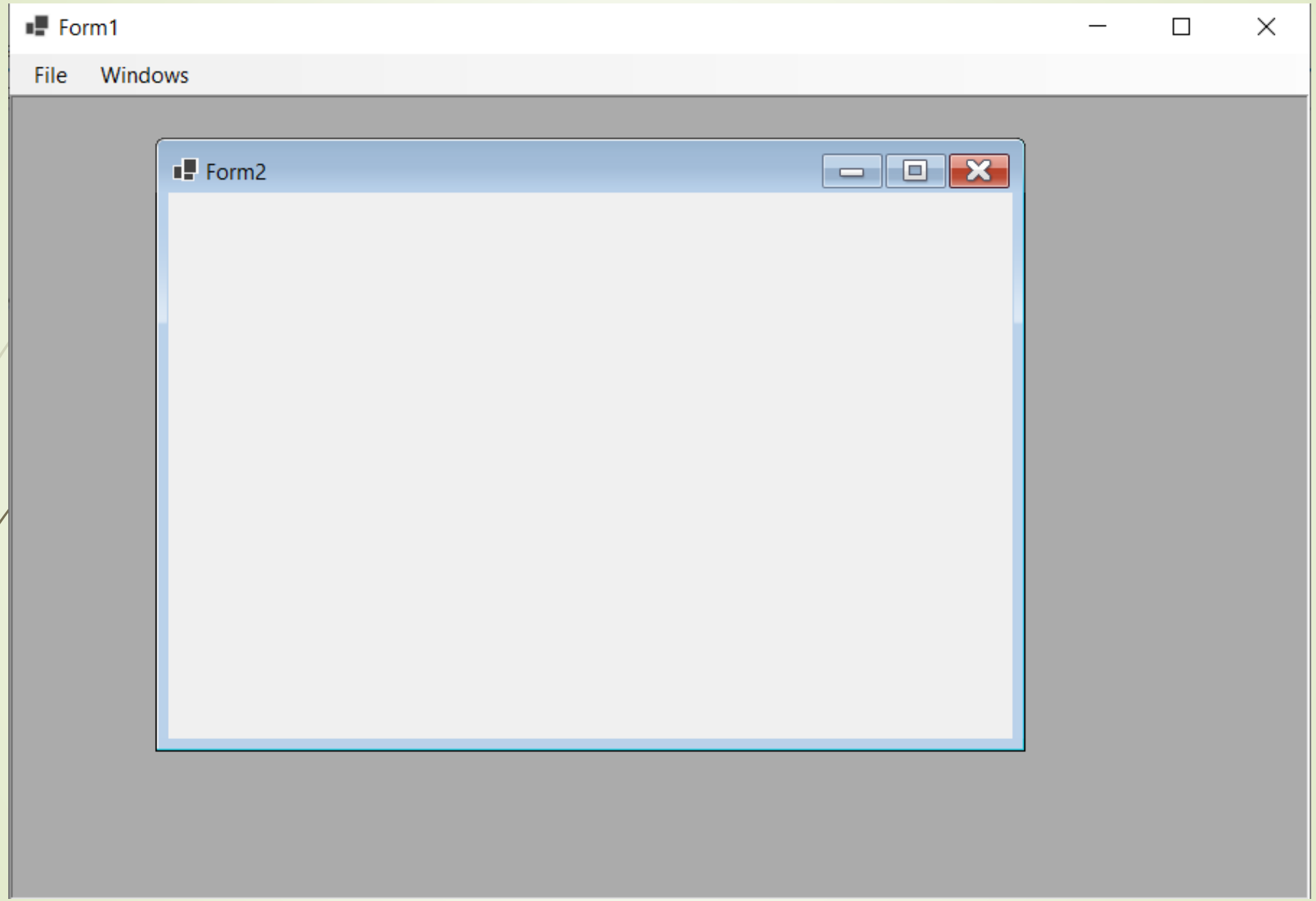


Output


File Edit View Git Project Build Debug Format Test Analyze Tools Extensions Window Help
Search
Toolbar_app
Form1
File Windows
Form2

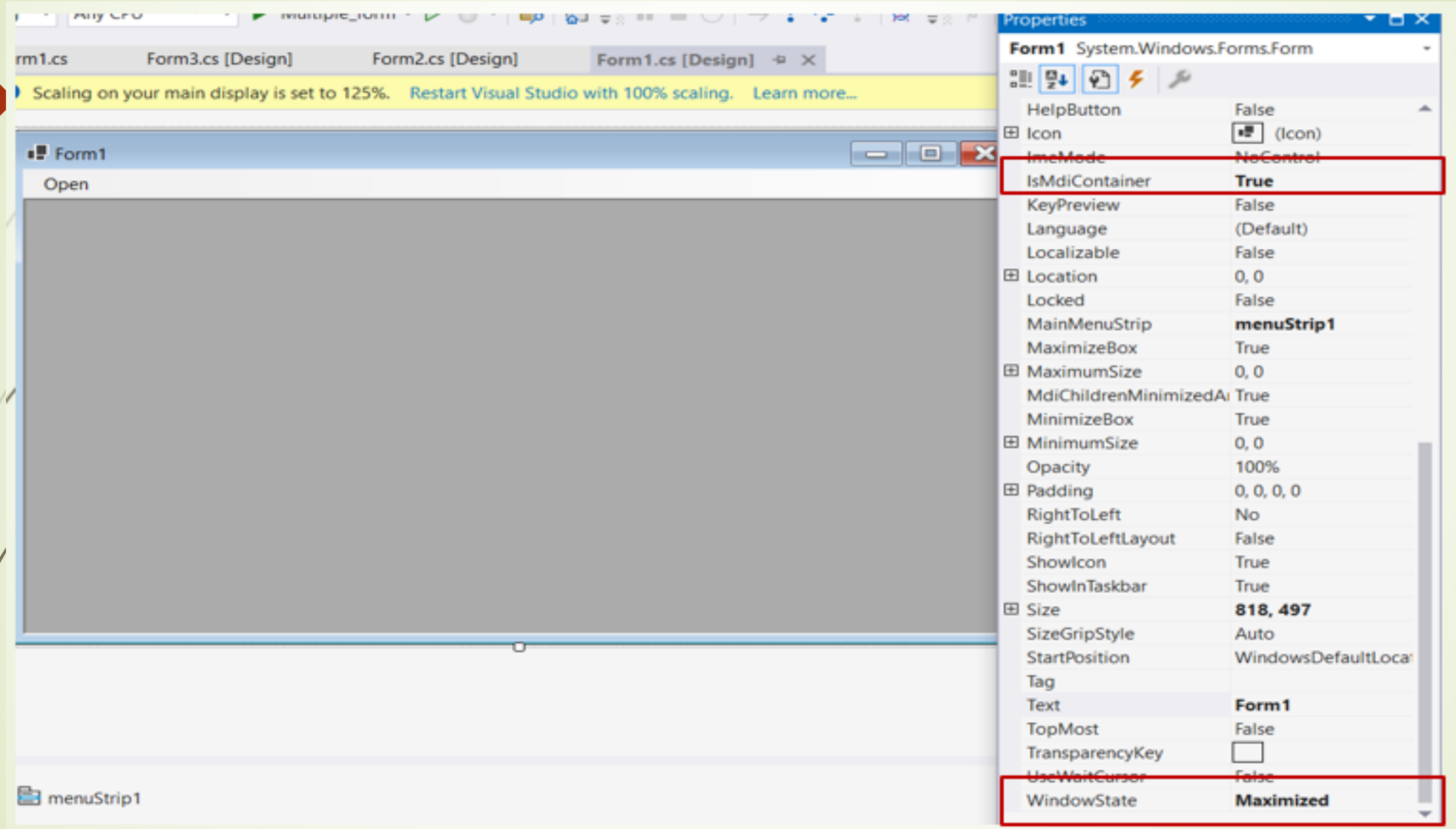
MdiParent Property Specifies the MDI parent form for the current form




Form1
File
Windows
Form2

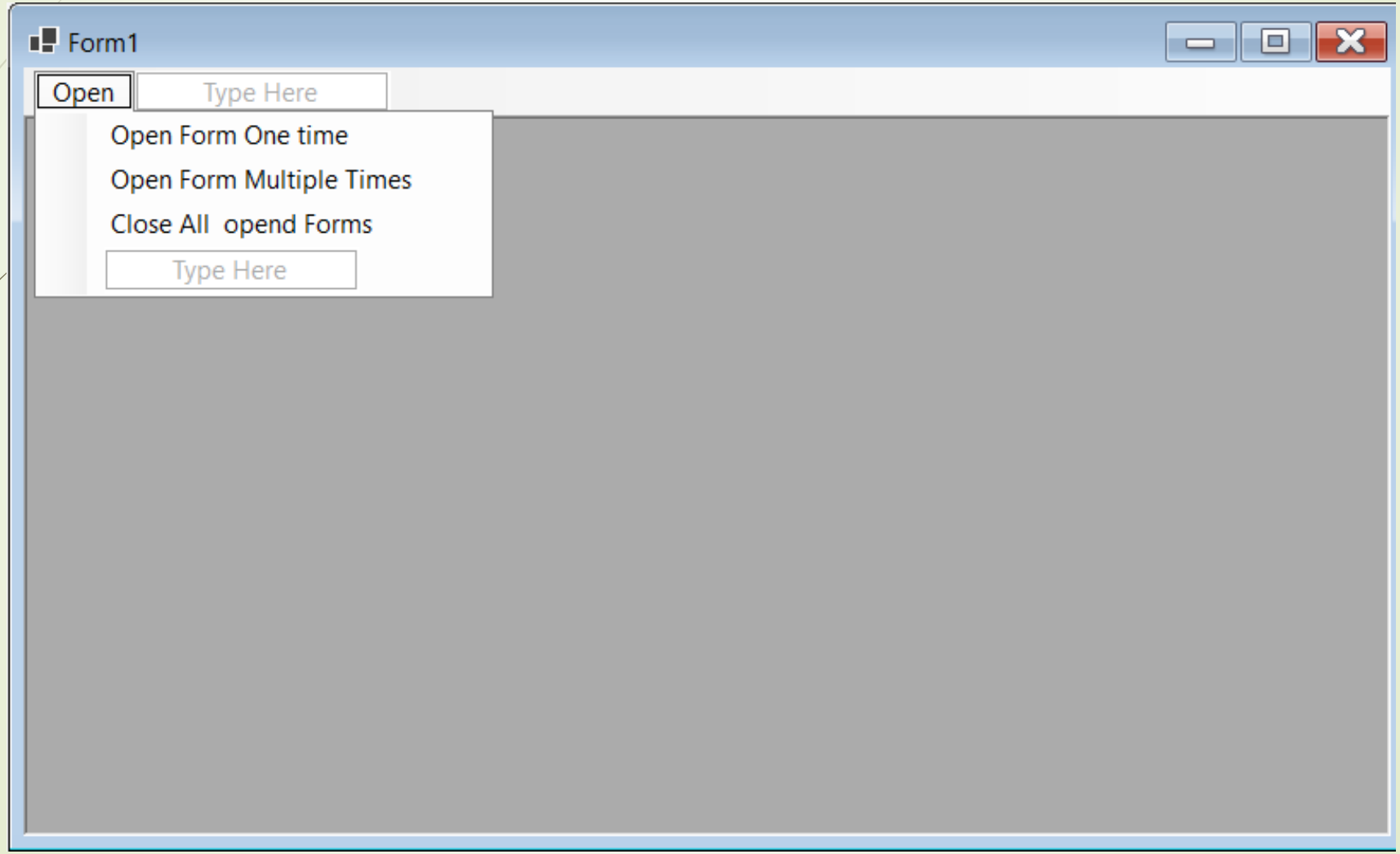
Example2

- Creating a MDI Parent ,(Create normal form & convert into MDI parent),
- Creating a MDI Child, (all other forms calling to MDI parent)
- Insert Menu strip in MDI Container and Insert some menu items,
- Open child form by clicking menu items as the following :
  - Form2 one time,
  - Form3 multiple times,
- Close All Opened Child forms by clicking menu item.
- Active form will not close.

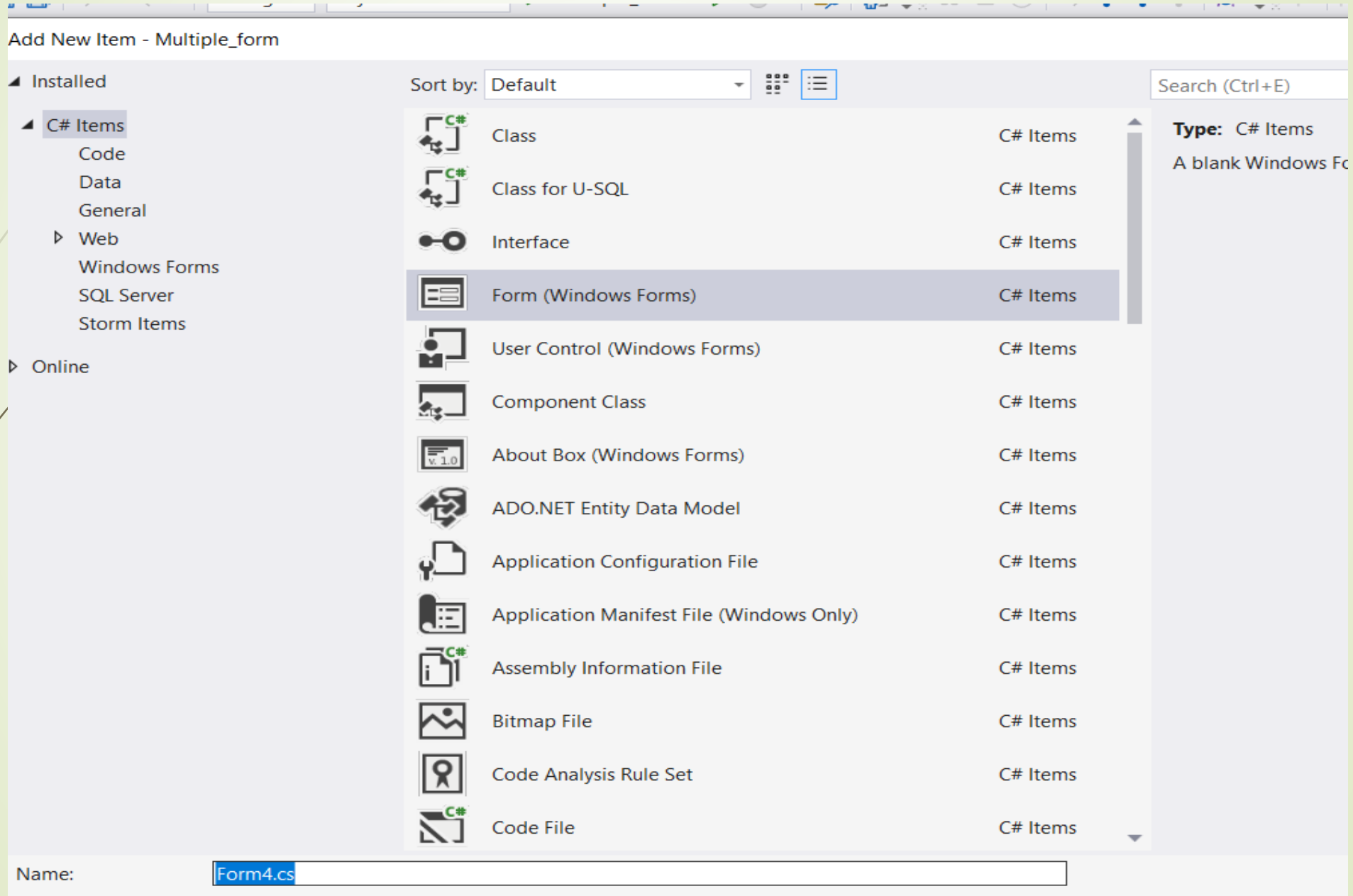


Change the form1 Properties as the following

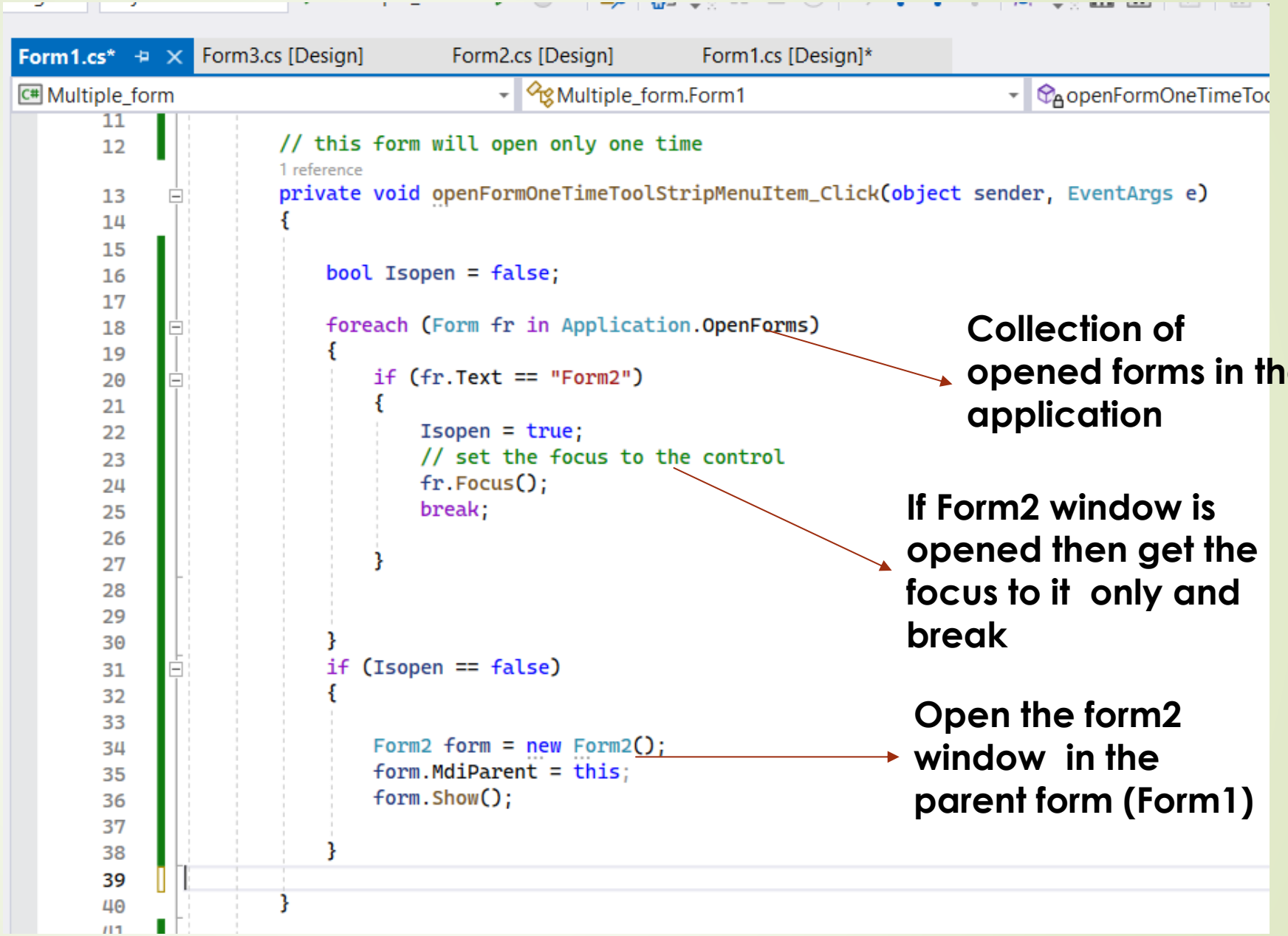
Add MenuStrips and add items as the following



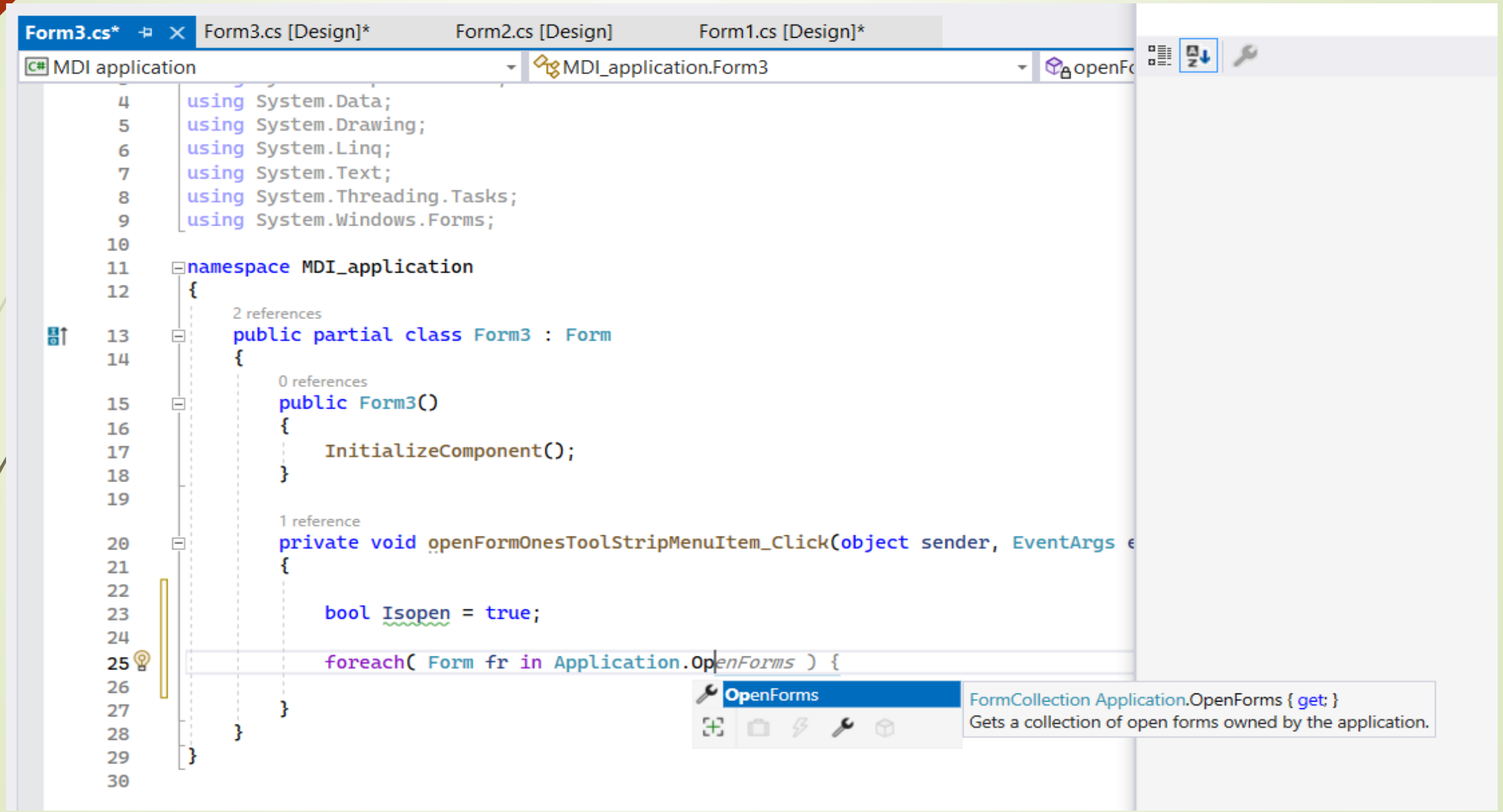
Add two new windows forms(Form2 and Form3)



Open the click event for the openFormOne item then add this code



Application.OpenForms



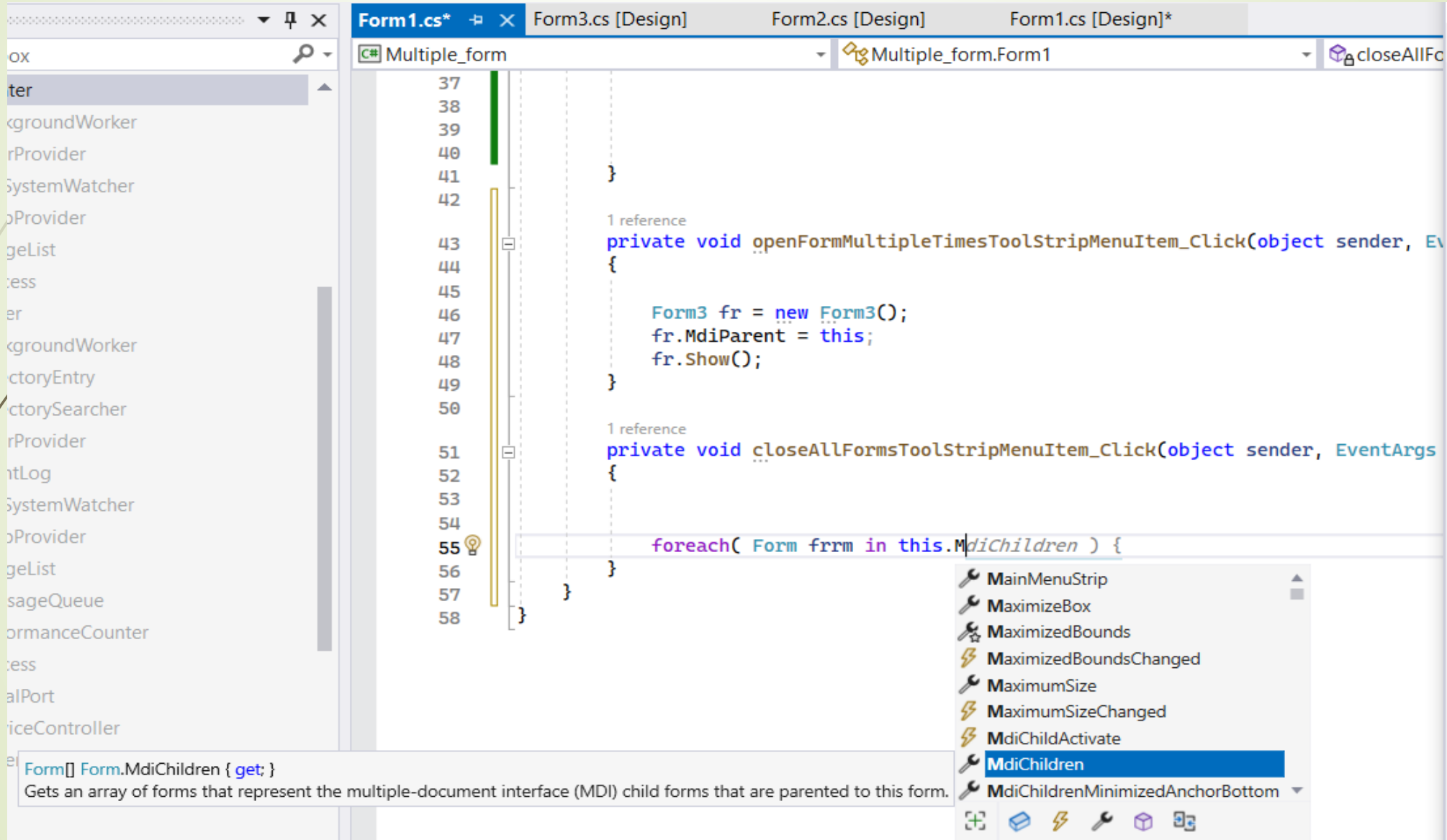
Open the click event for the openFormMultipleTimes item then add this code

```
}

1 reference
private void openFormMultipleTimesToolStripMenuItem_Click(object sender, EventArgs e)
{

    Form3 fr = new Form3();
    fr.MdiParent = this;
    fr.Show();
}

1 reference
```

Open the click event for the closeAllForms item then add this code

```
1 reference
private void closeAllFormsToolStripMenuItem_Click(object sender, EventArgs e)
{


    foreach( Form frrm in this.MdiChildren)
    {

        if(frrm.Focus() == true)
        {

            frrm.Visible = false;
            frrm.Close();
           // frrm.Dispose();

        }

    }
}
```

MdiChildren



Control.Focus Method

- The Focus method returns true if the control successfully received input focus.
- Selectable controls that can receive focus and show visual cues (like a blinking cursor, highlight, or selection border) when the user interacts with them.
- The control can have the input focus while not displaying any visual cues of having the focus. This behaviour is primarily observed by the no selectable controls listed below, or any controls derived from them.

Explanation

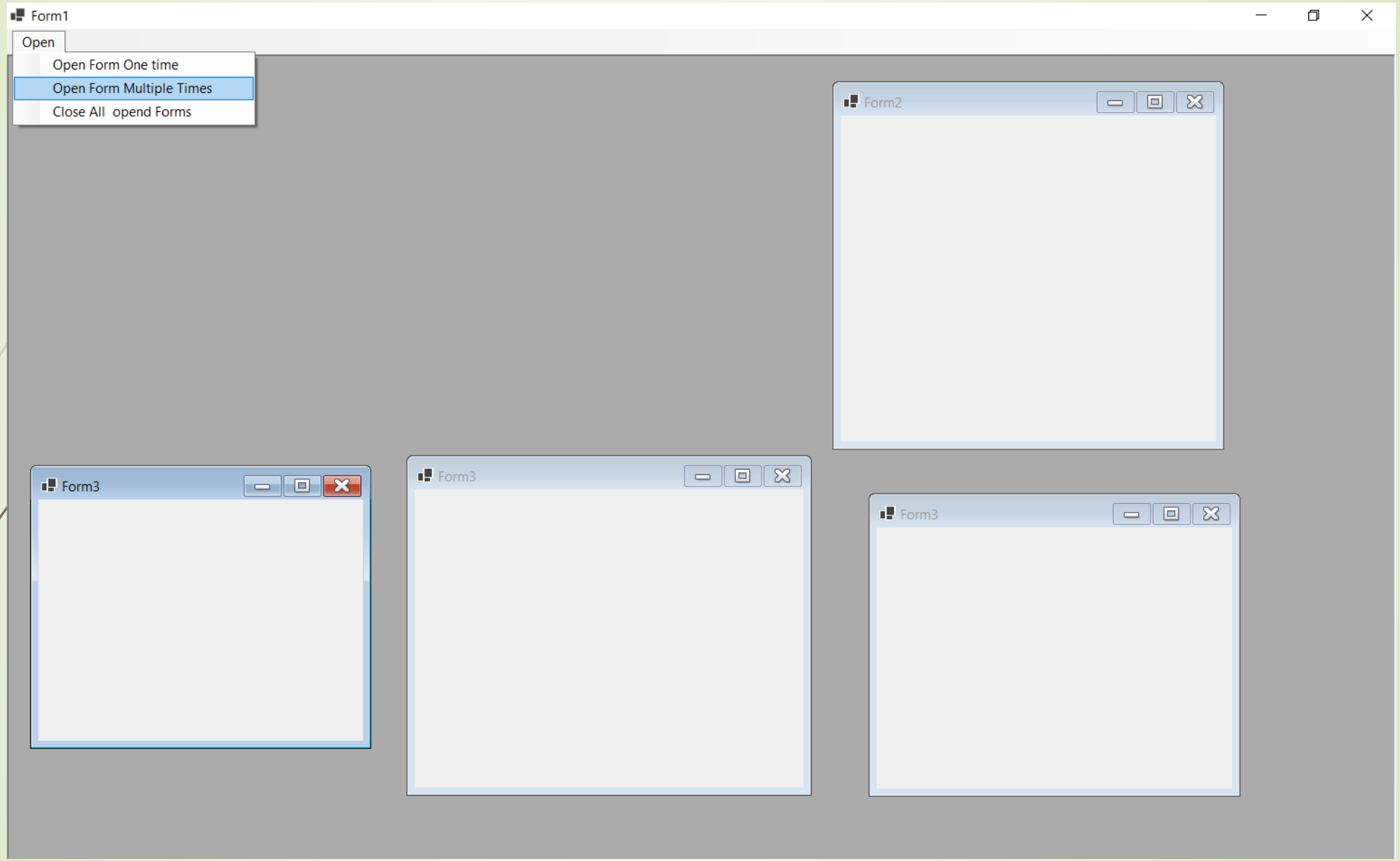
1. **Focus Method:**
   - In GUI programming (like Windows Forms), many controls (buttons, textboxes, etc.) can receive "input focus."
   - **Input focus** means that the control is ready to receive keyboard input.
2. **Return Value (`true` or `false`):**
   - If you call the `Focus()` method on a control:
     - It returns `true` → The control **successfully received focus**.
     - It returns `false` → The control **did not receive focus**.
3. **Focus Without Visual Cues:**
   - Normally, when a control has focus, you might see a blinking cursor or a highlighted border.
   - But **some controls** can have focus **without showing any visual change**.
   - Example: a `Label` or `Panel` usually doesn't highlight when focused.
4. **No Selectable Controls:**
   - The controls that don't show visual cues when focused are called "**non-selectable controls**".
   - Any controls derived from these (custom controls based on them) will behave the same way.

No Selectable Controls

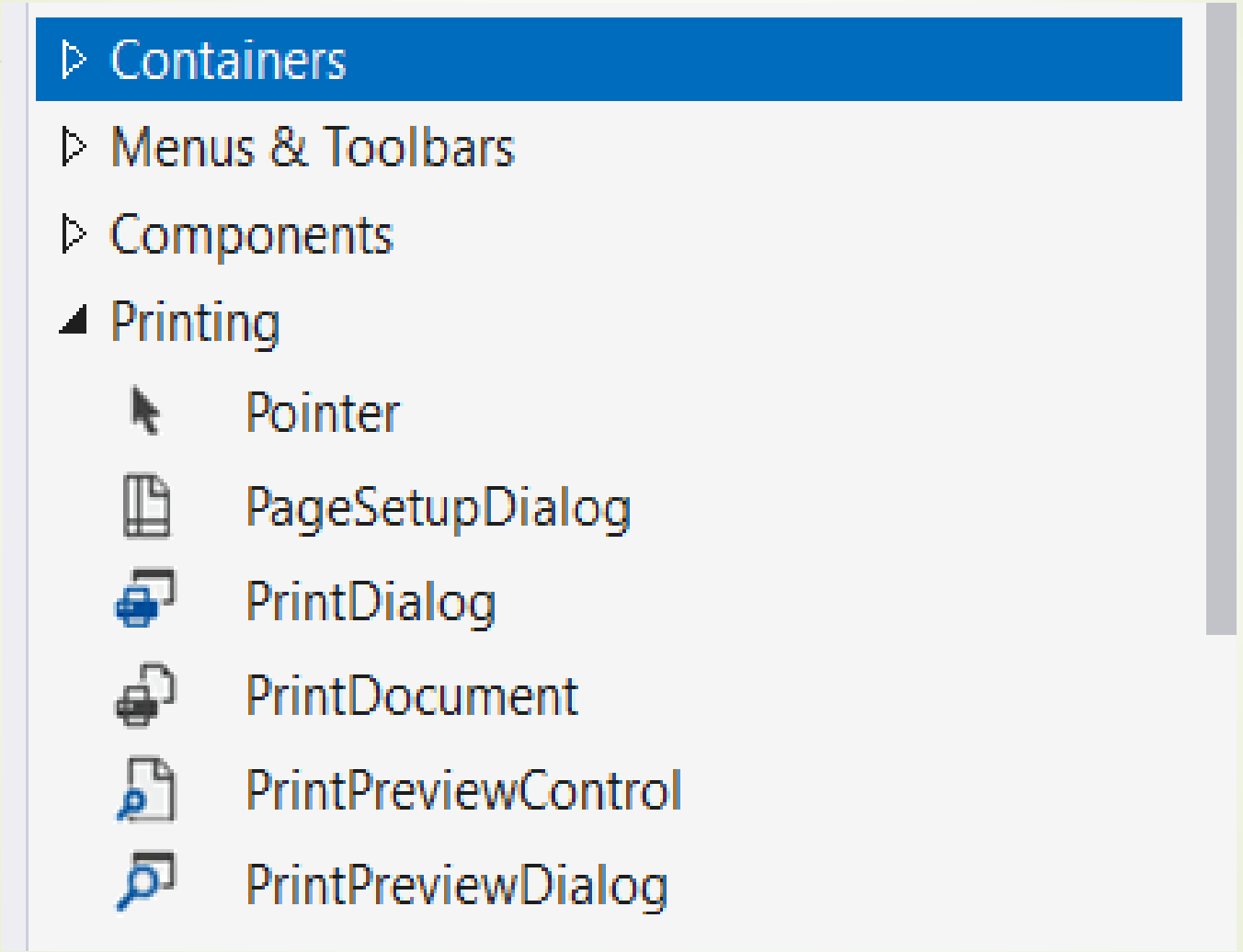
These controls cannot be selected by the user.

Selectable Controls

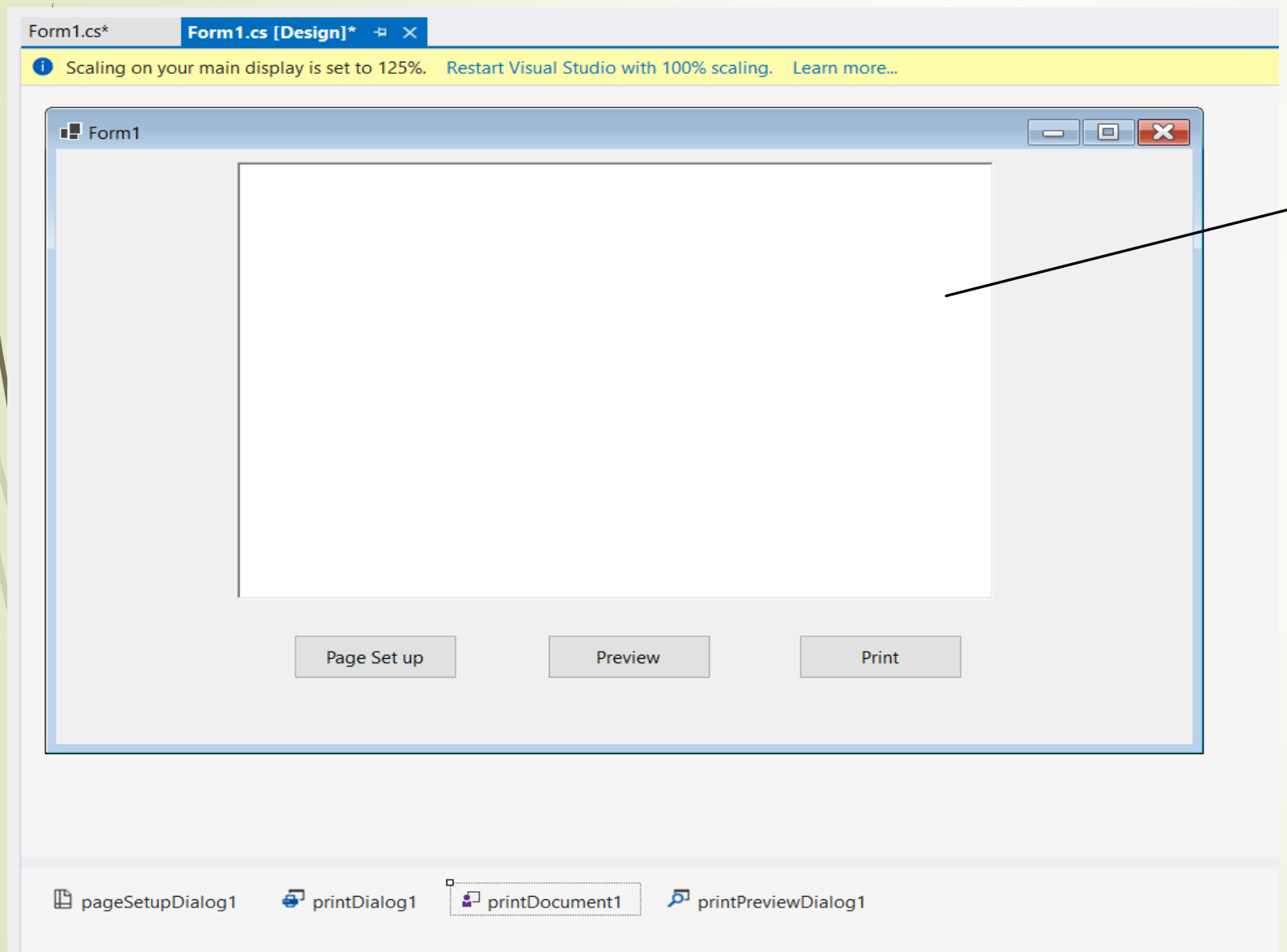
Control Type	Visual Cue / Behavior
TextBox	Shows a blinking cursor when focused
Button	Highlights or shows a border when clicked or focused
CheckBox	Shows a check mark when selected
RadioButton	Highlights the selected option
ComboBox / ListBox	Highlights the selected item
RichTextBox	Shows blinking cursor and highlights selected text


Form1
Open
Open Form One time
Open Form Multiple Times
Close All opend Forms
Form2
Form3
Form3
Form3

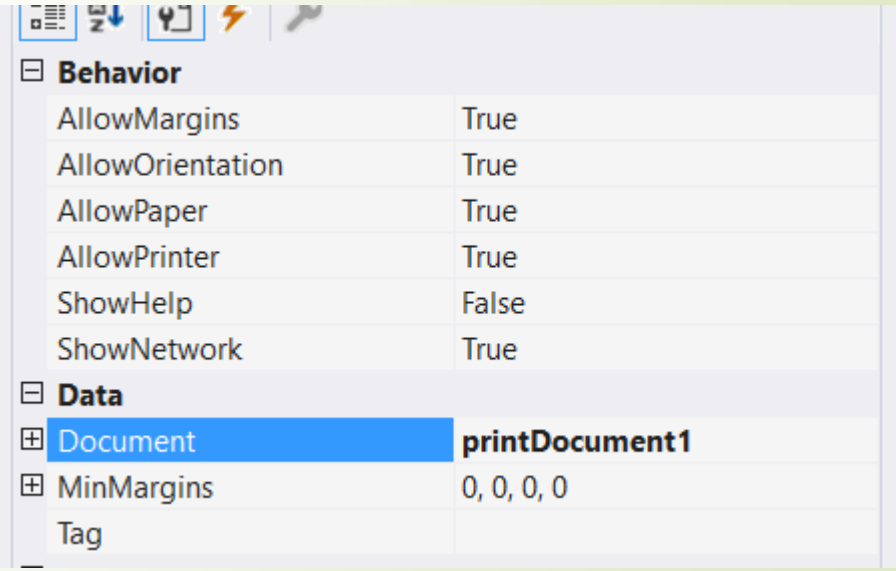
Printing tools



Make the following design then make the document property for the dialogs to **printDocumnet1**



RichTextBox



Open the **PrintPage** event and type the following code

Form1.cs* | Form1.cs [Design]*

printapplication | printapplication.Form1 | printDocument1_PrintPage(object sender, Pr

```
1  namespace printapplication
2  {
       4 references
3      public partial class Form1 : Form
4      {
           1 reference
5          public Form1()
6          {
7              InitializeComponent();
8          }
9
           1 reference
10         private void printDocument1_PrintPage(object sender, System.Drawing.Printing.PrintPageEventArgs
11         {
12
13             e.Graphics.DrawString(richTextBox1.Text, new Font("arial", 15), Brushes.AliceBlue, 20, 20);
14
15
16
17         }
18     }
19 }
```

Graphics class is for drawing shapes like arrows , arcs and so on

Print the string or text from the ritchTextBox1 with specific font and color

Location point(x,y) where the brush draw

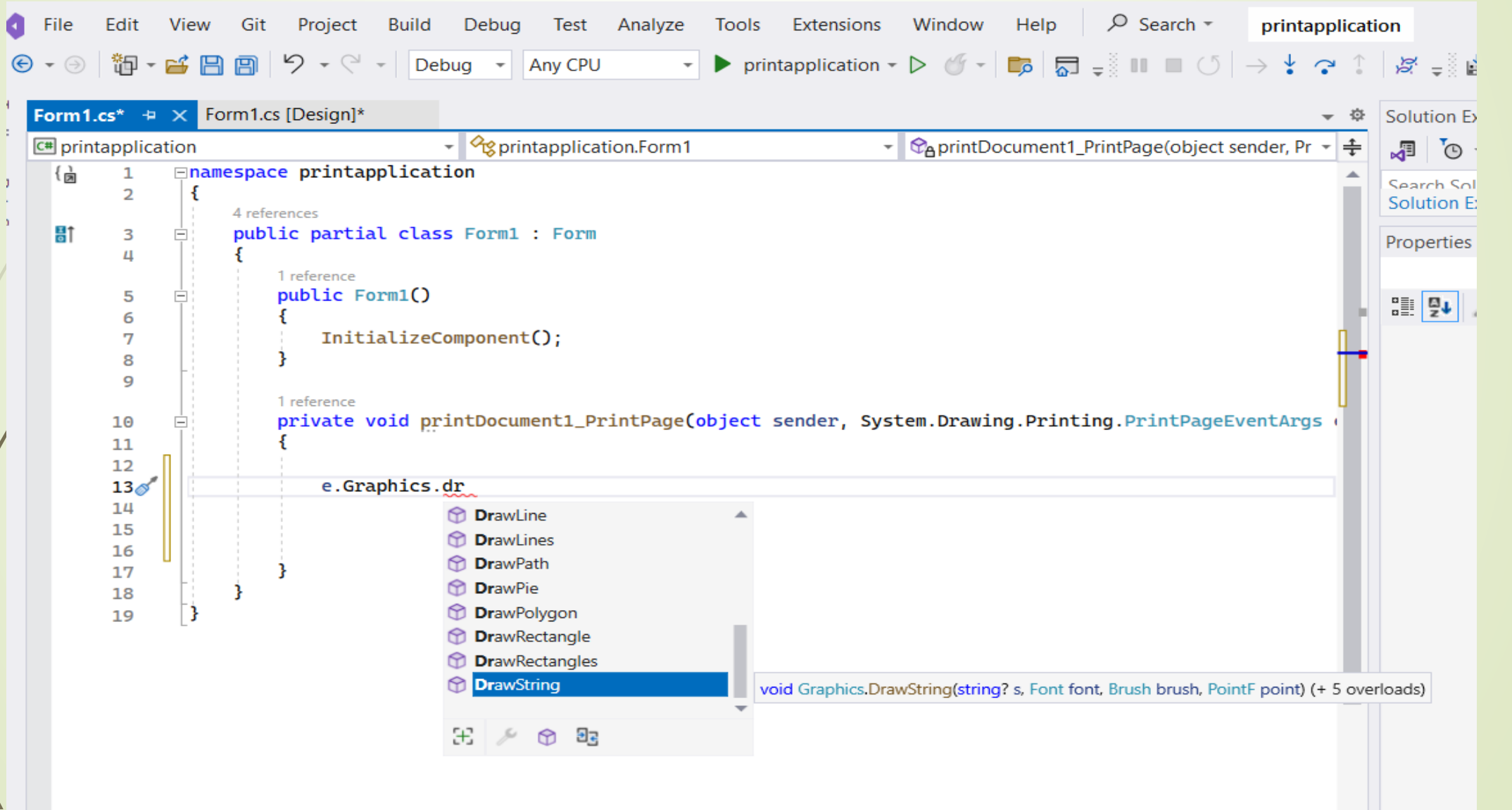
Remember the EventHandler and events

Whenever the mouse is being moved on top of a control, a mouse event is sent. This event is called **MouseMove** and is of type **MouseEventArgs**. It is initiated as follows:

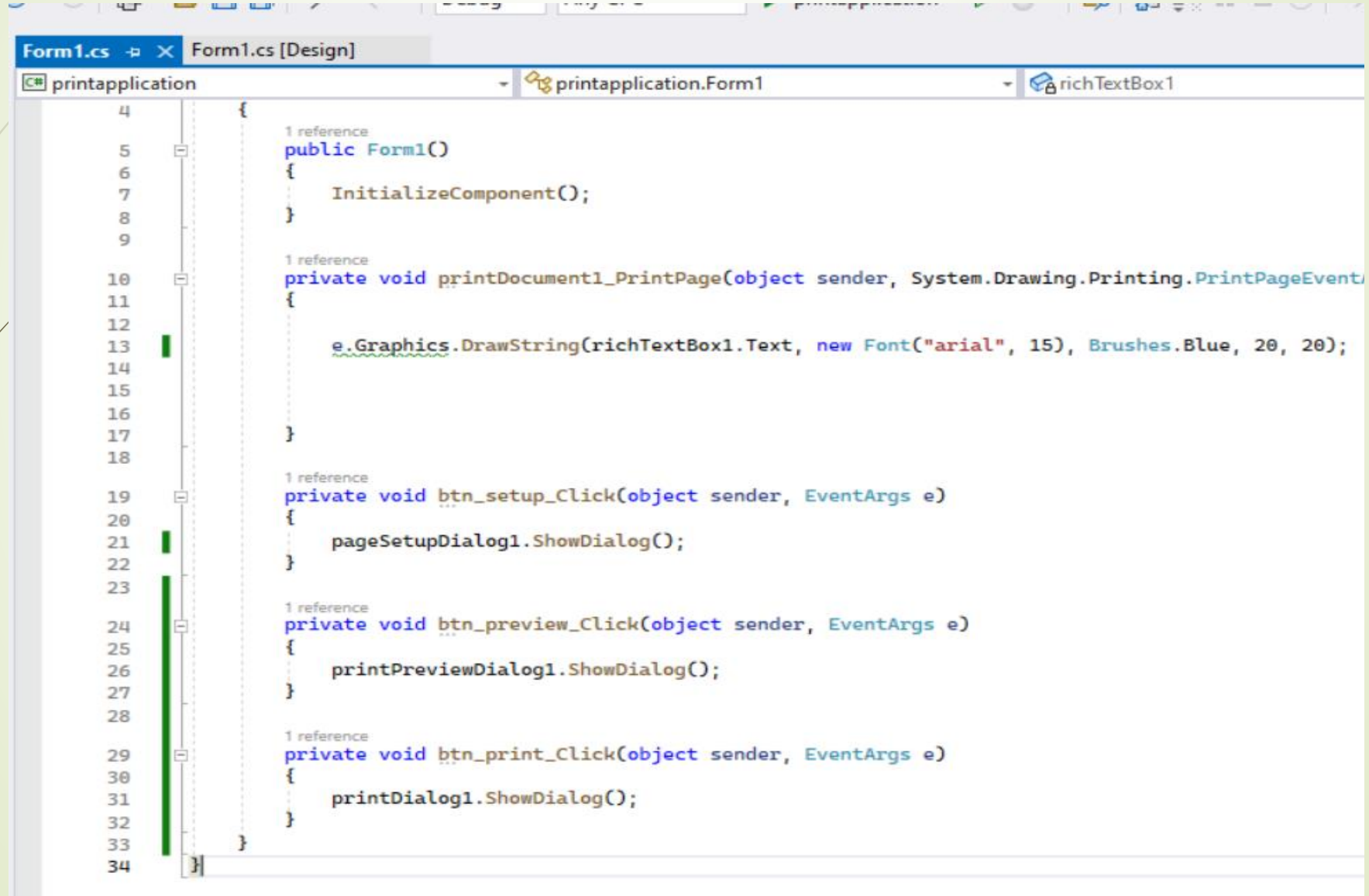
```
private void Control_MouseMove(object sender, System.Windows.Forms.MouseEventArgs e)
{

}
```

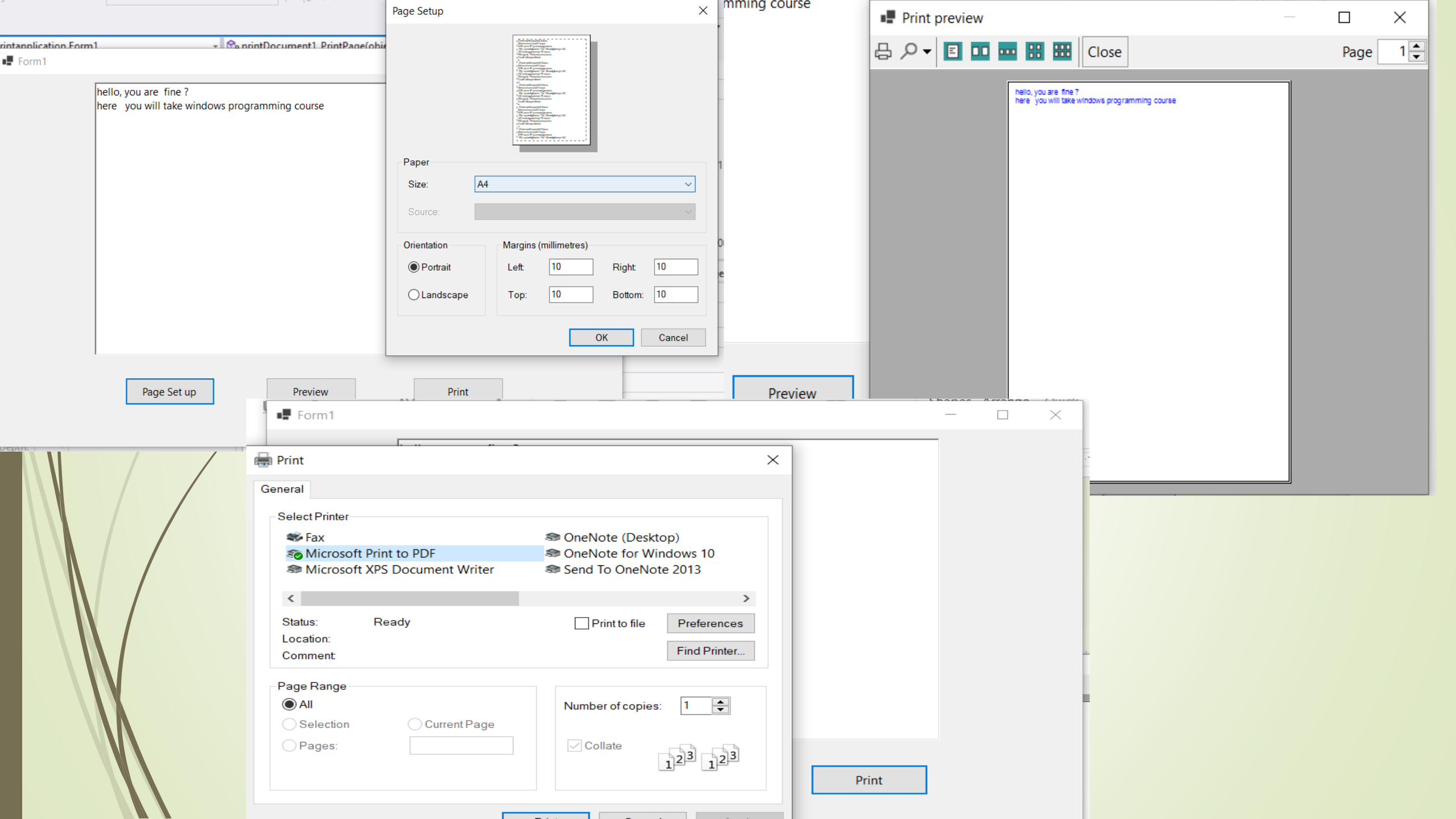
To implement this event, a **MouseEventArgs** argument is passed to the **MouseEventHandler** event implementer. The **MouseEventArgs** argument provides the necessary information about the event such as what button was clicked, how many times the button was clicked, and the location of the mouse.

DrawString() method print text with specified font type and color this method in Graphics class



Open the Click event for the pagesetup, preview and print buttons

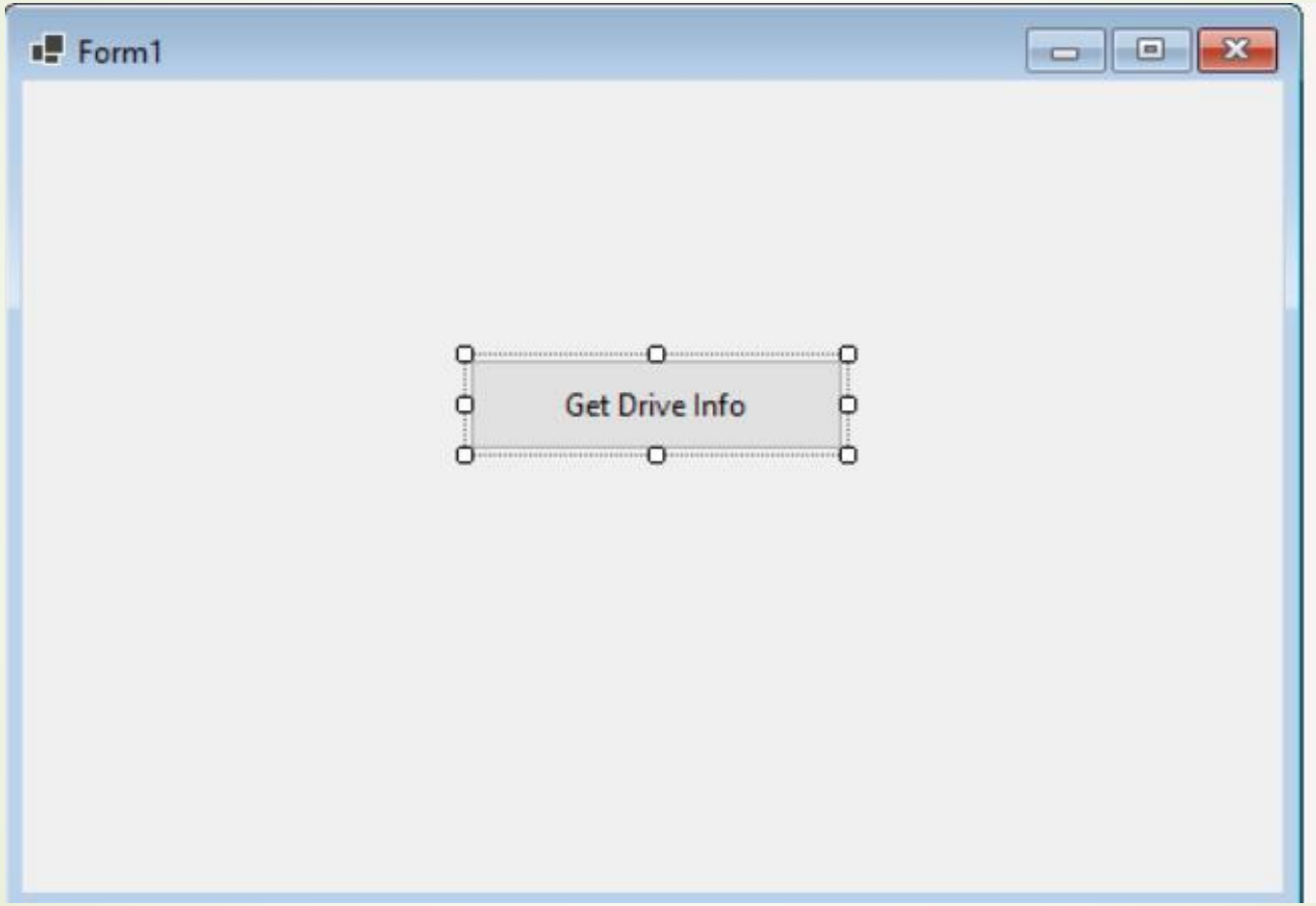



Page Setup
Paper
Size:
A4
Source:
Orientation
Portrait
Landscape
Margins (millimetres)
Left:
10
Right:
10
Top:
10
Bottom:
10
OK
Cancel
Print preview
Close
Page
1
hello, you are fine ?
here you will take windows programming course
Form1
hello, you are fine ?
here you will take windows programming course
Page Set up
Preview
Print
Preview
Form1
Print
General
Select Printer
Fax
Microsoft Print to PDF
Microsoft XPS Document Writer
OneNote (Desktop)
OneNote for Windows 10
Send To OneNote 2013
Status:
Ready
Location:
Comment:
Print to file
Preferences
Find Printer...
Page Range
All
Selection
Current Page
Pages:
Number of copies:
1
Collate
Print

DriveInfo Class

- Computer drive information is a useful feature to have in a program. We can use it to notify the user about insufficient disk space. We get drive information in C# with the DriveInfo Class. This class is located inside the System.IO namespace
- A single DriveInfo object represents one computer drive. We use the object's properties to get information from that drive. This way we look up its drive type, total size in bytes, drive name, root directory, and amount of free space.

Make the following design



In the click event handler put this code

Form1.cs* | Form1.cs [Design]*

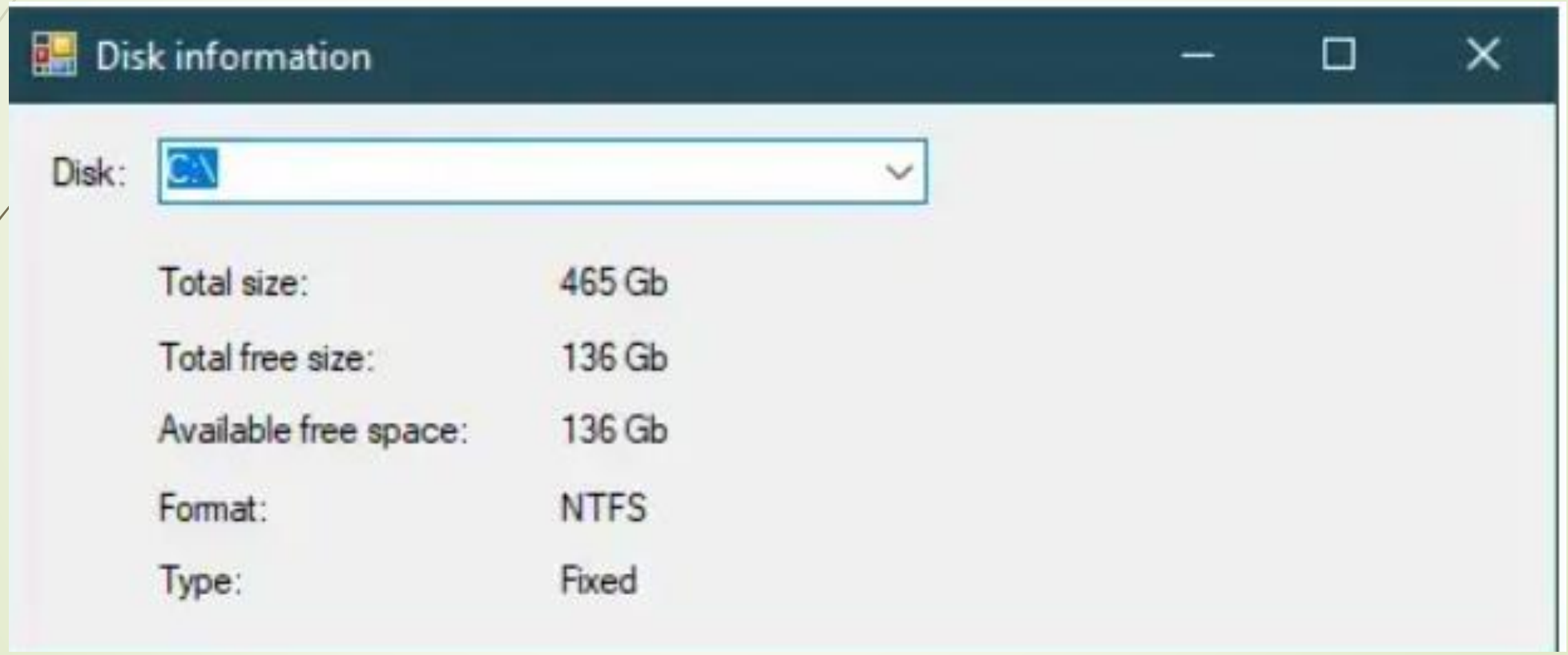
driveinfo | driveinfo.Form1 | btn_drive_Click(object sender, EventArgs e)

```
1   namespace driveinfo
2   {
        3 references
3       public partial class Form1 : Form
4       {
            1 reference
5           public Form1()
6           {
7               InitializeComponent();
8           }
9
            1 reference
10          private void btn_drive_Click(object sender, EventArgs e)
11          {
12
13              DriveInfo drive = new DriveInfo("C:");
14              double total = drive.TotalSize;
15              double free_space = drive.AvailableFreeSpace;
16              double res = free_space / total * 100;
17              MessageBox.Show("the free space in % is :" + res + "%");
18
19              
20
21
22          }
23      }
24  }
```

Explanation

1. `DriveInfo drive = new DriveInfo("C:");`
   - Accesses information about the C: drive on your computer.
2. `double total = drive.TotalSize;`
   - Gets the total storage of the drive in bytes.
3. `double free_space = drive.AvailableFreeSpace;`
   - Gets the free storage available to the user in bytes.
4. `double res = free_space / total * 100;`
   - Calculates the percentage of free space.
5. `MessageBox.Show("the free space in % is :" + res + "%");`
   - Shows a pop-up message displaying the free space percentage.

Task_1: create the following window and get the disk Information



References

- Developing and Implementing Windows Based Applications with Microsoft Visual C# .NET and Microsoft Visual Studio.
- https://learn.microsoft.com/en-/dotnet/api/system.windows.forms.control.focus?view=windowsdesktop-8.0
- https://learn.microsoft.com/en-us/dotnet/desktop/winforms/advanced/multiple-document-interface-mdi-applications?view=netframeworkdesktop-4.8
- https://kodify.net/csharp/computer-drive/get-drive-info/